

1. 背景与动机

1.1 高维数据与维度灾难

在当代机器学习与数据科学中，研究者与工程师常常面对高维观测数据。以图像为例，一张 28×28 的灰度手写数字图像对应 784 维的向量，而 32×32 的彩色图像则对应 3072 维。语音或时间序列数据由于采样率高，常常达到数千至数万甚至更高的维度；文本数据采用词袋模型或上下文嵌入时，也会形成数百到上千维的向量。

高维度带来的直接影响包括计算成本与存储成本的上升，更严重的是“维度灾难”（curse of dimensionality）：随着维度增加，数据分布稀疏，样本需求指数级增长，基于距离度量的方法（如最近邻）逐渐失效，模型容易陷入过拟合而缺乏泛化能力。

1.2 传统降维方法的局限

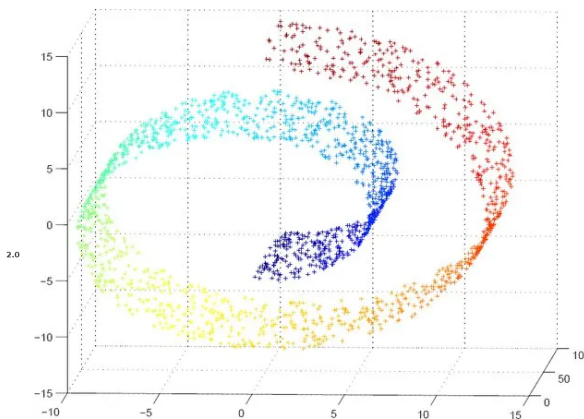
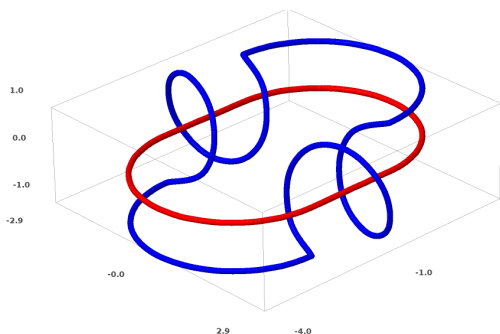
应对高维问题的经典方法包括主成分分析（Principal Component Analysis, PCA）和线性判别分析（Linear Discriminant Analysis, LDA）等。它们通过线性映射来提取数据的主方向或判别方向，从而实现降维与去噪。然而，这类方法存在明显局限：许多实际数据（如图像、语音、文本）在高维空间中呈现复杂的非线性结构，例如弯曲的流形或簇状分布，而线性方法无法充分捕捉这些非线性特征。因此，需要能够学习非线性表示的工具，自编码器（Autoencoder, AE）便是在此背景下被提出并广泛采用。

1.3 流形假设与非线性表示学习

自编码器的理论基础与“流形假设”（manifold hypothesis）密切相关。该假设认为：尽管数据位于高维空间 $\mathbb{R}^d$ ，但它们实际上近似分布在一个低维流形上。可表示为：

$$\mathbf{x} = g(\mathbf{z}), \quad \mathbf{z} \in \mathbb{R}^m, m \ll d,$$

其中 $\mathbf{z}$ 是潜在的低维因子，映射 g 将其映射到观测空间。若假设成立，那么学习从 $\mathbf{x}$ 到 $\mathbf{z}$ 的编码映射以及从 $\mathbf{z}$ 到 $\mathbf{x}$ 的解码映射，就成为一种合理的建模策略。自编码器正是通过参数化的编码器与解码器来学习这种关系，从而实现非线性降维、表示学习与重构。



1.4 低维表示应当具备的性质

从工程实践出发，理想的低维表示应具备以下性质：

- **紧凑性**：用更少的维度表达原始信息。
- **判别性**：不同类别或簇在潜在空间中更容易分离。
- **稳健性**：对输入噪声或微小扰动保持稳定。
- **连贯性**：潜在空间中小幅移动对应输入空间中有意义的变化。

自编码器通过灵活的结构设计（深度编码器、非线性激活、卷积结构等）和正则化手段（稀疏约束、去噪机制等），为获得这些性质提供了可能。

1.5 应用场景

- 数据可视化
 - 降维至二维或三维后可直接绘制潜在空间分布。例如 MNIST 手写数字在二维潜在空间中常形成清晰的簇，帮助研究者理解数据结构。
- 数据压缩
 - 通过低维编码显著降低存储与传输开销。例如将 MNIST 的 784 维数据压缩至 32 维，压缩倍率达到 $784/32 \approx 24.5$ 。
- 聚类
 - 在编码器输出的低维表示上进行聚类，通常优于直接在高维原始空间操作，能得到更高质量的簇划分。
- 异常检测
 - 利用重构误差作为指标：模型能较好地重构训练分布上的样本，而对分布外样本则常产生较大误差，从而实现异常检测。
- 生成模型与数据增强
 - 概率化的扩展（如变分自编码器）可以在潜在空间采样并生成新样本，用于图像生成、数据增强等任务。
- 推荐系统
 - 在用户-物品交互数据中，自编码器能够学习低维潜在表示，用于预测评分与推荐，被广泛应用于深度协同过滤。

1.6 学习范式

从学习范式上看，自编码器属于自监督学习：训练目标不依赖人工标注，而是利用输入本身构造监督信号（例如“输入等于输出”或“带噪输入映射到干净输出”）。

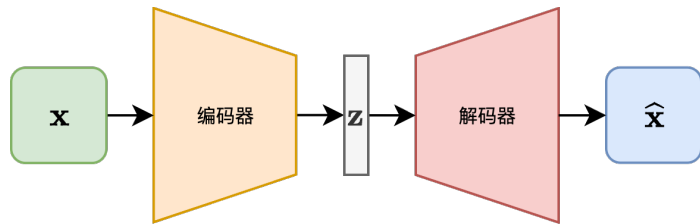
这种特性使得自编码器能够充分利用无标签数据，先进行大规模预训练，再在少量有标注数据上进行微调，服务于下游任务。

2. 自编码器的基本结构

2.1 基本思想

自编码器（Autoencoder, AE）是一类无监督神经网络模型，其目标是在输入与输出之间建立一种近似恒等的映射。换句话说，它的训练目标并不是预测标签，而是尽可能从压缩后的表示中重建出原始输入，即输入 $\mathbf{x}$ 经压缩与解码后得到的输出 $\hat{\mathbf{x}}$ 应尽可能接近原始输入。

与多层感知机（MLP）相比，自编码器的特殊之处在于：其输出空间与输入空间相同，但网络中部存在一个**瓶颈层**（bottleneck）。瓶颈层的（有效）维度通常远小于输入维度，这迫使网络在有限表示能力下提取输入数据的核心特征。这一低维的中间表示称为**潜在表征**（latent representation），它往往蕴含数据的语义或结构信息。



2.2 网络结构

标准自编码器由两个对称的部分组成：**编码器**（encoder）和**解码器**（decoder）。

(1) 编码器

编码器负责将输入映射到潜在空间，给定 $\mathbf{x} \in \mathbb{R}^d$ ：

$$\mathbf{z} = f_{\theta}(\mathbf{x})$$

其中 f 表示任意符合要求的神经网络组件， θ 为编码器参数集合。

(2) 解码器

解码器试图从潜在表示 $\mathbf{z}$ 重建出原始输入：

$$\hat{\mathbf{x}} = g_{\phi}(\mathbf{z})$$

其中 g 为任意符合要求的神经网络组件， ϕ 为解码器参数集合。

于是，整个自编码器就可表示为一个复合映射：

$$\hat{\mathbf{x}} = g_{\phi}(f_{\theta}(\mathbf{x})).$$

在实践中，编码器和解码器往往由多层网络堆叠而成，形成**深度自编码器**，以捕捉更加复杂的非线性结构。瓶颈层起到信息压缩的作用，是自编码器学习有效表示的关键所在。此外，编码器和解码器可以是任意结构，例如 CNN 自编码器用于图像，RNN 自编码器用于序列。Transformer 也是自编码器中常用的结构。

2.3 训练目标与损失函数

自编码器的训练目标是最小化输入与输出之间的差异。最常用的重建误差衡量方法有以下两类：

(1) 均方误差 (MSE)

适用于连续数值型数据，如灰度图像：

$$L(\mathbf{x}, \hat{\mathbf{x}}) = \|\mathbf{x} - \hat{\mathbf{x}}\|_2^2 = \sum_{i=1}^d (\mathbf{x}_i - \hat{\mathbf{x}}_i)^2.$$

(2) 交叉熵损失

适用于输入为0-1向量或概率分布的情况：

$$L(\mathbf{x}, \hat{\mathbf{x}}) = - \sum_{i=1}^d [\mathbf{x}_i \log \hat{x}_i + (1 - \mathbf{x}_i) \log(1 - \hat{x}_i)].$$

更一般地，自编码器的优化问题可表述为：

$$\min_{\theta, \phi} \mathbb{E}_{\mathbf{x} \sim \mathcal{D}} L(\mathbf{x}, g_{\phi}(f_{\theta}(\mathbf{x}))).$$

2.4 训练流程

自编码器的训练过程与普通神经网络类似，但其监督信号完全来自输入数据本身。具体步骤如下：

- **前向传播**：输入 $\mathbf{x}$ 经编码器得到潜在表示 $\mathbf{z} = f_{\theta}(\mathbf{x})$ ，再经解码器得到重建 $\hat{\mathbf{x}} = g_{\phi}(\mathbf{z})$ 。
- **损失计算**：根据 $L(\mathbf{x}, \hat{\mathbf{x}})$ 衡量重建误差。
- **反向传播**：计算损失对参数 θ, ϕ 的梯度。
- **参数更新**：通过优化算法（如 SGD、Adam）更新参数。

在实践中，常采用 mini-batch 梯度下降，并辅以正则化方法（权重衰减、Dropout、稀疏约束等），以避免过拟合或学到“恒等映射”。特别地，当瓶颈层维度大于输入维度，或网络过于复杂时，自编码器可能退化为简单的复制器，从而失去特征学习能力。这时需要通过维度约束或加入噪声来提高表示的有效性。

2.5 与线性降维的关系

自编码器与传统的线性降维方法——**主成分分析 (Principal Component Analysis, PCA)**——有着紧密联系。若编码器和解码器均为线性映射，且采用 MSE 损失，则自编码器的最优解等价于 PCA。

(1) 主成份分析

主成分分析是一种经典的降维方法，它通过寻找数据中“方差最大”的方向来提取最重要的低维表示。这使得我们既能压缩数据，又能尽可能保留主要的信息。

设样本矩阵为

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}^{(1)\top} \\ \mathbf{x}^{(2)\top} \\ \vdots \\ \mathbf{x}^{(N)\top} \end{bmatrix} \in \mathbb{R}^{N \times d}, \quad \mathbf{x}^{(i)} \in \mathbb{R}^d,$$

并假设数据已做零均值化 ($\tilde{\mathbf{x}}^{(i)} = \mathbf{x}^{(i)} - \frac{1}{N} \sum_{j=1}^N \mathbf{x}^{(j)}$)。

PCA 的目标是寻找一个维度为 $d \times k$ 的正交投影矩阵

$$\mathbf{W} \in \mathbb{R}^{d \times k}, \quad \mathbf{W}^\top \mathbf{W} = \mathbf{I}_k,$$

使得 k 维投影表示

$$\mathbf{z}^{(i)} = \mathbf{W}^\top \mathbf{x}^{(i)} \in \mathbb{R}^k$$

能够最大程度保留数据的信息。

投影后的 k 维表示可重建为：

$$\hat{\mathbf{x}}^{(i)} = \mathbf{W} \mathbf{z}^{(i)} = \mathbf{W} \mathbf{W}^\top \mathbf{x}^{(i)},$$

目标是最小化均方重建误差：

$$\min_{\mathbf{W}} \frac{1}{N} \sum_{i=1}^N \|\mathbf{x}^{(i)} - \hat{\mathbf{x}}^{(i)}\|_2^2 = \min_{\mathbf{W}} \frac{1}{N} \|\mathbf{X} - \mathbf{X} \mathbf{W} \mathbf{W}^\top\|_F^2.$$

主成分分析的求解：

- **目标函数：** 目标是最小化数据点 $\mathbf{x}^{(i)}$ 与其重建点 $\hat{\mathbf{x}}^{(i)} = \mathbf{W} \mathbf{W}^\top \mathbf{x}^{(i)}$ 之间的均方距离：

$$\frac{1}{N} \sum_{i=1}^N (\mathbf{x}^{(i)} - \mathbf{W} \mathbf{W}^\top \mathbf{x}^{(i)})^\top (\mathbf{x}^{(i)} - \mathbf{W} \mathbf{W}^\top \mathbf{x}^{(i)})$$

- **展开目标函数：**

$$\frac{1}{N} \sum_{i=1}^N \left[\mathbf{x}^{(i)\top} \mathbf{x}^{(i)} - 2 \mathbf{x}^{(i)\top} \mathbf{W} \mathbf{W}^\top \mathbf{x}^{(i)} + \mathbf{x}^{(i)\top} \mathbf{W} \mathbf{W}^\top \mathbf{W} \mathbf{W}^\top \mathbf{x}^{(i)} \right]$$

- **利用正交性简化：** 由于 $\mathbf{W}$ 是正交矩阵（即 $\mathbf{W}^\top \mathbf{W} = \mathbf{I}$ ），其中 $\mathbf{W}$ 是 $d \times k$ 矩阵（ $k < d$ ），满足 $\mathbf{W}^\top \mathbf{W} = \mathbf{I}_k$ ，但 $\mathbf{W} \mathbf{W}^\top \neq \mathbf{I}_d$ 。

$$\mathbf{x}^{(i)\top} \mathbf{W} \mathbf{W}^\top \mathbf{W} \mathbf{W}^\top \mathbf{x}^{(i)} = \mathbf{x}^{(i)\top} \mathbf{W} (\mathbf{W}^\top \mathbf{W}) \mathbf{W}^\top \mathbf{x}^{(i)} = \mathbf{x}^{(i)\top} \mathbf{W} \mathbf{I} \mathbf{W}^\top \mathbf{x}^{(i)} = \mathbf{x}^{(i)\top} \mathbf{W} \mathbf{W}^\top \mathbf{x}^{(i)}$$

目标函数简化为：

$$\frac{1}{N} \sum_{i=1}^N \left[\mathbf{x}^{(i)\top} \mathbf{x}^{(i)} - \mathbf{x}^{(i)\top} \mathbf{W} \mathbf{W}^\top \mathbf{x}^{(i)} \right]$$

- **等价最大化问题：** 最小化上述目标函数等价于最大化：

$$\frac{1}{N} \sum_{i=1}^N \mathbf{x}^{(i)\top} \mathbf{W} \mathbf{W}^\top \mathbf{x}^{(i)}$$

约束条件为 $\mathbf{W}^\top \mathbf{W} = \mathbf{I}$ 。

- **迹的性质应用：** 利用迹的循环置换性质：

$$\mathbf{x}^{(i)\top} \mathbf{W} \mathbf{W}^\top \mathbf{x}^{(i)} = \text{Tr}(\mathbf{x}^{(i)\top} \mathbf{W} \mathbf{W}^\top \mathbf{x}^{(i)}) = \text{Tr}(\mathbf{W}^\top \mathbf{x}^{(i)} \mathbf{x}^{(i)\top} \mathbf{W})$$

因此有：

$$\sum_{i=1}^N \mathbf{x}^{(i)\top} \mathbf{W} \mathbf{W}^\top \mathbf{x}^{(i)} = \text{Tr} \left(\mathbf{W}^\top \left(\sum_{i=1}^N \mathbf{x}^{(i)} \mathbf{x}^{(i)\top} \right) \mathbf{W} \right)$$

- **协方差矩阵：** 假设数据已中心化（均值为零），则 $\frac{1}{n} \sum_{i=1}^n \mathbf{x}_i \mathbf{x}_i^\top$ 是协方差矩阵 Σ 。因此，目标函数变为：

$$\max_{\mathbf{W}} \text{Tr}(\mathbf{W}^\top \Sigma \mathbf{W}) \quad \text{subject to} \quad \mathbf{W}^\top \mathbf{W} = \mathbf{I}$$

- **最优解：** 当 $\mathbf{W}$ 的列是协方差矩阵 Σ 的前 k 个最大特征值对应的特征向量时， $\text{Tr}(\mathbf{W}^\top \Sigma \mathbf{W})$ 达到最大值。
也就是说，数据矩阵 $\mathbf{X}$ 的主成分方向通过对协方差矩阵

$$\Sigma = \frac{1}{N-1} \mathbf{X}^\top \mathbf{X}$$

做特征分解来实现，其特征向量就是主成分方向，特征值对应每个方向上的方差。

实际上，这个过程与对数据矩阵本身做奇异值分解（SVD）是等价的：若 $\mathbf{X} = \mathbf{U} \mathbf{S} \mathbf{V}^\top$ ，则协方差矩阵可以写作

$$\Sigma = \frac{1}{N-1} \mathbf{V} \mathbf{S}^\top \mathbf{S} \mathbf{V}^\top,$$

可知，协方差矩阵的特征向量正好是右奇异向量 $\mathbf{V}$ ，特征值由奇异值平方除以 $N-1$ 给出，而左奇异向量 $\mathbf{U}$ 与奇异值 $\mathbf{S}$ 的乘积则表示每个样本在这些主成分方向上的投影。

也就是说，通过 SVD 无需显式计算协方差矩阵，就可以直接得到主成分方向和样本的低维表示。这也是实践中计算 PCA 时常用的方法。

(2) 线性自编码器

假设编码器为：

$$\mathbf{z} = \mathbf{W} \mathbf{x}, \quad \mathbf{W} \in \mathbb{R}^{k \times d},$$

解码器为

$$\hat{\mathbf{x}} = \mathbf{W}^\top \mathbf{z} = \mathbf{W}^\top \mathbf{W} \mathbf{x}.$$

目标函数为

$$L(\mathbf{x}, \hat{\mathbf{x}}) = \|\mathbf{x} - \mathbf{W}^\top \mathbf{W} \mathbf{x}\|_2^2.$$

令数据矩阵为 $\mathbf{X} \in \mathbb{R}^{N \times d}$ ，协方差矩阵为 $\Sigma = \frac{1}{N} \mathbf{X}^\top \mathbf{X}$ 。可以证明，最优的 $\mathbf{W}$ 张成的空间，恰好是 Σ 的前 k 个特征向量所对应的子空间，即 PCA 的主成分子空间。

也就意味着，线性自编码器等于 PCA。非线性自编码器则是 PCA 的推广，因而能够捕捉更加复杂的非线性流形。

(3) 实例

02-linear-ae_vs_pca.py

3. 自编码器的主要实现类型

上一节介绍了自编码器的基本结构以及通过最小化重建误差来学习潜在表示的训练目标。然而，自编码器很容易面临一个问题：模型在某些情况下可能退化为恒等映射，仅仅记忆输入而不真正学习特征。因此，研究者们提出了多种解决办法，通过结构约束、正则化以及噪声干预等机制来提升自编码器的表示能力。本节将介绍三类典型的实现类型，即不完备自编码器、稀疏自编码器和去噪自编码器。这三种方法分别利用瓶颈限制、稀疏激活和抗噪训练来克服“平凡解”，从而使自编码器在特征提取、可视化和生成建模中发挥更大作用。

3.1 不完备自编码器（Undercomplete Autoencoder）

不完备自编码器是最基础的一种形式，其核心思想是通过限制潜在空间的维度，迫使模型仅能保留输入数据中最重要的部分。设输入为向量

$$\mathbf{x} \in \mathbb{R}^d,$$

则潜在表示为

$$\mathbf{z} \in \mathbb{R}^k, \quad k < d.$$

这种约束形成了一个瓶颈结构（bottleneck），类似于在信息传递的过程中设置狭窄通道，网络必须学习提炼输入中的关键信息以通过瓶颈，而无法简单地逐元素复制。

形式上，编码器与解码器分别表示为

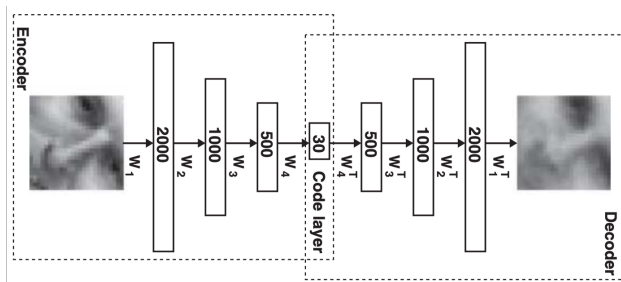
$$\mathbf{z} = f_{\theta}(\mathbf{x}), \quad \hat{\mathbf{x}} = g_{\phi}(\mathbf{z}),$$

其中 f_{θ} 与 g_{ϕ} 通常由多层感知机或卷积网络构成。训练目标是最小化输入与重建之间的误差，即

$$\min_{\theta, \phi} \mathbb{E}_{\mathbf{x} \sim \mathcal{D}} \|\mathbf{x} - g_{\phi}(f_{\theta}(\mathbf{x}))\|^2.$$

当网络完全由线性层构成且损失函数为平方误差时，上述优化问题的解等价于主成分分析（PCA）。模型学习到的潜在空间就是数据协方差矩阵前 k 个特征向量所张成的子空间。引入非线性后，自编码器能够学习比 PCA 更复杂的非线性流形结构，从而显著增强表示能力。

在实际应用中，这类自编码器常用于降维与可视化。例如在 MNIST 数据集中，原始 28×28 灰度图像被展开为 784 维向量，通过一个具有低维度瓶颈维度的自编码器可以有效压缩信息。当将潜在空间维度降至 2 或 3 时，不同类别的数字往往自动形成聚类，从而为无监督学习中的模式发现提供了直观手段。



3.2 稀疏自编码器 (Sparse Autoencoder)

尽管不完备自编码器通过降低潜在维度在很大程度上避免了恒等映射的平庸解，但这种约束并不总是充分的。当网络容量过大时，即使 $k < d$ ，模型也可能学到接近复制的映射。为了进一步约束潜在空间的使用方式，人们提出了稀疏自编码器，它不依赖于潜在维度的大小，而是通过引入稀疏性约束，使一定比例的潜在单元保持0输出，仅少量单元被激活，从而限制潜在表征的信息传输能力。

稀疏性的思想来源于神经科学的观察：在大脑皮层中，面对特定刺激时往往只有极少数神经元被激活。将这一思想引入自编码器，即便潜在空间维度较大，对于每个输入样本，只有少量单元会被激活，从而强迫模型以更加选择性的方式编码信息。

设第 j 个潜在单元在整个训练集上的平均激活为：

$$\hat{\rho}_j = \frac{1}{N} \sum_{i=1}^N f_{\theta}(\mathbf{x}^{(i)})_j,$$

我们希望它接近一个预设的稀疏比例 $\rho \ll 1$ ，例如 $\rho = 0.05$ 。这一约束可以通过在损失函数中加入 KL 散度实现：

$$\Omega = \sum_{j=1}^k \text{KL}(\rho \parallel \hat{\rho}_j) = \sum_{j=1}^k \left[\rho \log \frac{\rho}{\hat{\rho}_j} + (1 - \rho) \log \frac{1 - \rho}{1 - \hat{\rho}_j} \right].$$

于是训练目标变为

$$\min_{\theta, \phi} \mathbb{E}_{\mathbf{x} \sim \mathcal{D}} \|\mathbf{x} - g_{\phi}(f_{\theta}(\mathbf{x}))\|^2 + \beta \Omega,$$

其中超参数 β 控制稀疏正则化的强度。

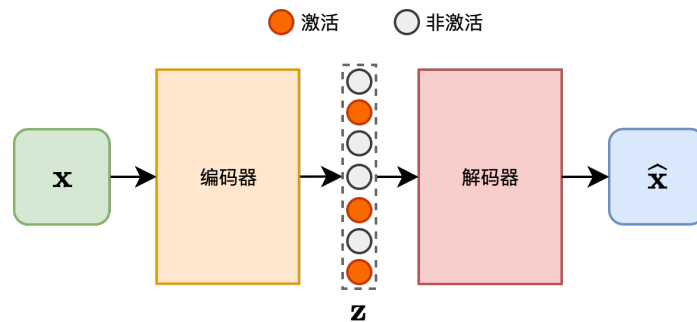
除了 KL 散度正则，一种更简单的方法是接在潜在表示 $\mathbf{z}$ 上施加 L_1 范数正则化，即将损失函数改为

$$\min_{\theta, \phi} \mathbb{E}_{\mathbf{x} \sim \mathcal{D}} \|\mathbf{x} - g_{\phi}(f_{\theta}(\mathbf{x}))\|^2 + \lambda \|f_{\theta}(\mathbf{x})\|_1,$$

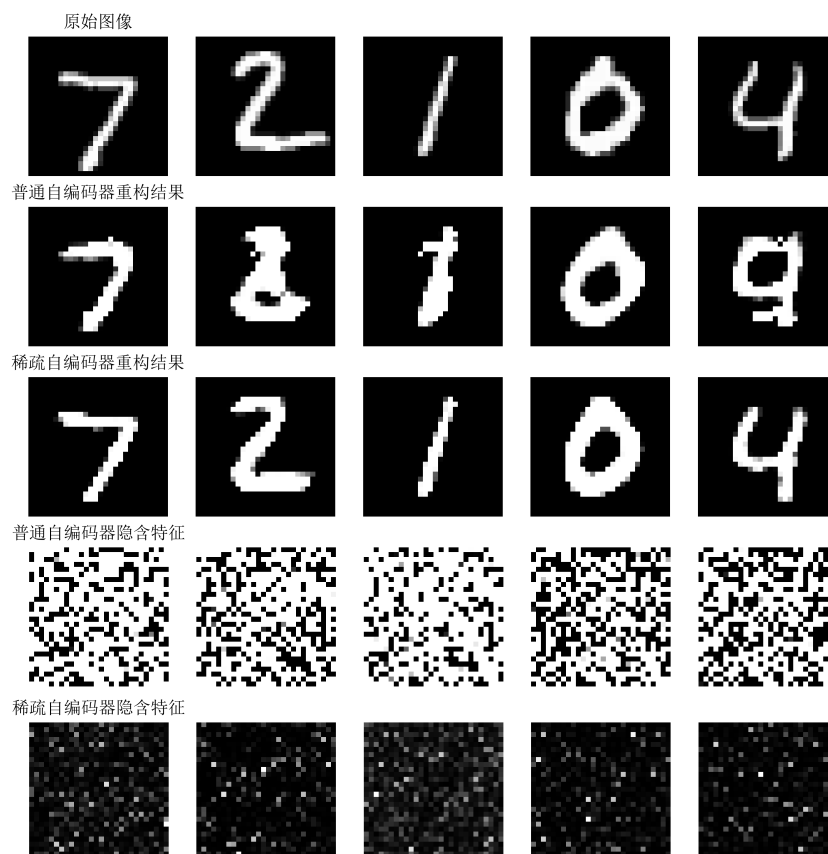
其中 λ 控制稀疏性强度。由于 L_1 正则化会鼓励部分分量为零值，因此这种方法同样可以实现潜在表示的稀疏性，并且不需要统计平均激活。

KL 散度和 L_1 正则都可以实现潜在表示的稀疏性，但侧重点略有不同。KL 散度正则直接约束每个潜在单元在整个训练集上的平均激活，使稀疏性更精确且可控，适合希望严格设定稀疏比例的场景；而 L_1 正则化则是在每个样本的潜在向量上施加稀疏约束，操作简单且易于实现，但稀疏性可能随样本而波动。总体而言，两种方法都能迫使模型以少量激活单元表达信息，从而提升特征的选择性与可解释性，实际应用中可根据具体任务和实现便利性选择合适方案。

稀疏性赋予了模型更强的特征学习能力，即便潜在维度较大，信息仍需通过少数激活单元进行表达，从而避免了退化为记忆机制的风险。在自然图像的任务中，人们发现稀疏自编码器学到的基向量往往类似于边缘或纹理检测器。这种结果与生物视觉皮层的神经元响应模式高度一致，因此稀疏自编码器不仅在机器学习中有价值，也为理解神经计算提供了重要启示。

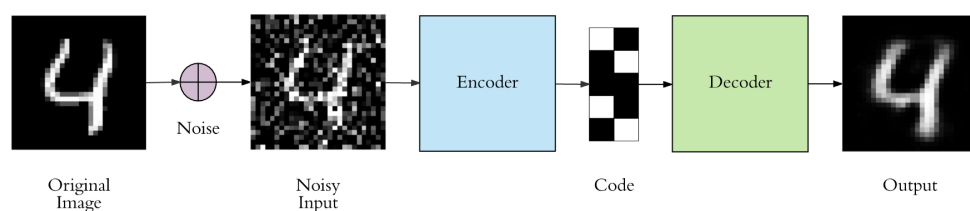


实例: 03-sparse_ae.py



3.3 去噪自编码器 (Denoising Autoencoder, DAE)

不完备自编码器和稀疏自编码器都是通过对潜在空间的约束来提升表示能力，而去噪自编码器则采取了另一种思路：直接在输入上施加扰动，迫使模型学习对噪声和部分缺失数据具有鲁棒性的表示。其核心理念是，即使输入被部分破坏，模型仍应能够恢复出原始的干净数据，从而获得更加稳定和可迁移的特征。



具体来说，给定输入 $\mathbf{x}$ ，首先通过某种破坏机制将其转换为带噪版本 $\tilde{\mathbf{x}}$ ，例如加入高斯噪声或随机遮蔽部分维度。编码器随后将 $\tilde{\mathbf{x}}$ 映射到潜在表示

$$\mathbf{z} = f_{\theta}(\tilde{\mathbf{x}}),$$

解码器再根据 $\mathbf{z}$ 重建输出

$$\hat{\mathbf{x}} = g_{\phi}(\mathbf{z}).$$

训练目标是最小化重建结果与原始输入之间的差异，即

$$\min_{\theta, \phi} \mathbb{E}_{\mathbf{x} \sim \mathcal{D}, \tilde{\mathbf{x}} \sim q(\tilde{\mathbf{x}}|\mathbf{x})} L(\mathbf{x}, g_{\phi}(f_{\theta}(\tilde{\mathbf{x}}))),$$

其中 $q(\tilde{\mathbf{x}}|\mathbf{x})$ 表示输入的破坏机制，常见形式包括高斯噪声

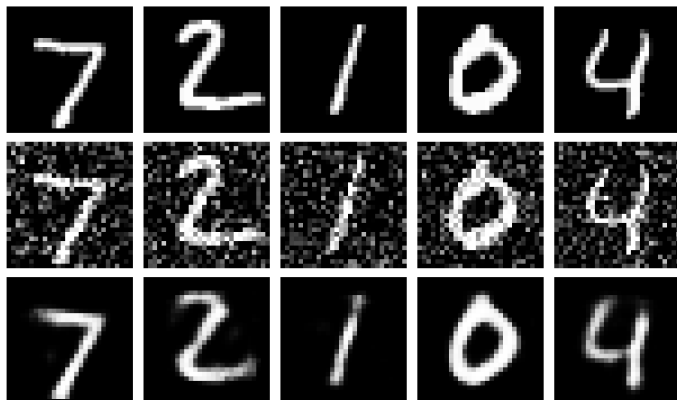
$$\tilde{\mathbf{x}} = \mathbf{x} + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma^2 \mathbf{I}),$$

以及掩蔽噪声，即随机选取部分分量置为零。损失函数 L 通常取平方误差或交叉熵。

在某些任务或输入类型下，直接对原始输入施加噪声可能不太合适，例如离散数据、文本序列或敏感特征。此时，去噪自编码器可以通过多种方式实现扰动而不破坏输入结构。常见方法包括：对输入的嵌入或连续表示施加高斯噪声或 dropout；对离散或符号型输入进行随机替换、删除或遮蔽；利用数据增强生成扰动样本，如图像的旋转、平移、遮挡或模糊，保证语义不变的同时增加抗噪能力；或者直接在潜在表示 $\mathbf{z}$ 上加入噪声，再通过解码器重建原始输入。无论具体实现如何调整，其核心思想始终是一致的：迫使模型在面对不完整或扰动信息时仍能恢复原始目标，从而学习更加鲁棒和稳定的特征表示。

与前述自编码器不同，去噪自编码器不依赖潜在维度的瓶颈或稀疏性约束，而是通过抗噪训练迫使模型学习稳定的特征。这种稳健性使得学到的表示在下游任务中更具可迁移性。例如，在图像去噪实验中，即便输入图像被随机遮挡或加入椒盐噪声，去噪自编码器仍能恢复接近原始的清晰图像。此外，这一思想也为后续更复杂的生成模型提供了启发，如变分自编码器（VAE）和扩散模型，它们在处理噪声和生成高质量样本方面发挥了重要作用。

• 实例：04-denoise_ae.py



4. 自编码器的应用

自编码器最初被提出时，主要用于降维与特征学习。随着模型的发展，它已经被广泛应用于数据分析、模式识别、异常检测、推荐系统、图像处理与生成等多个领域。下面我们将详细介绍其中的典型应用，并结合具体案例说明其作用。

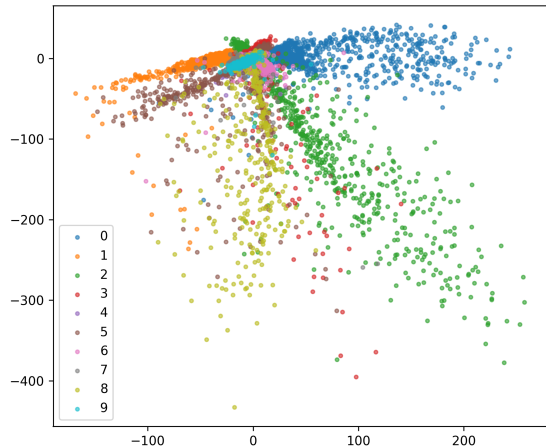
4.1 降维

自编码器可以被视为主成分分析（PCA）的非线性推广，因此其最直接的应用就是降维。

例：MNIST 数据可视化

在 MNIST 手写数字数据集（784 维）上训练一个瓶颈层为 2 维的不完备自编码器，得到的潜在空间可以直接在二维平面可视化。结果往往显示出不同数字在二维平面上的明显分簇，这说明自编码器能够自动学习区分数字类别所需的特征，即使在训练过程中并没有使用任何标签信息。

这种降维能力在数据可视化和特征提取中非常有用。例如在基因表达数据中，可以用自编码器将上万维的基因特征压缩到几十维，以便进一步分析样本的分布与模式。



• 代码：01-ae_2d_mnist.py

4.2 聚类

由于自编码器能够学习到紧凑且有意义的潜在表示，这些表示天然适合作为聚类的输入。因此，自编码器常与 K-means、谱聚类、层次聚类等方法结合。

(1) k均值聚类

聚类是一种典型的无监督学习方法，其目标是将未标注的数据划分为若干组，使得同一组内的数据点尽可能相似，而不同组之间的差异尽可能大。k 均值聚类（k-means clustering）是最经典的聚类方法之一。它的核心思想是通过寻找若干“质心”（centroids），将每个样本划分到距离其最近的质心所在的簇中，并不断迭代更新簇的划分与质心的位置，直至达到稳定状态。

设数据集为：

$$\mathbf{X} = \{\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots, \mathbf{x}^{(n)}\}, \quad \mathbf{x}^{(i)} \in \mathbb{R}^d,$$

目标是将其划分为 k 个簇 $C_1, C_2, \dots, C_k$ 。每个簇 C_j 的中心定义为：

$$\boldsymbol{\mu}_j = \frac{1}{|C_j|} \sum_{\mathbf{x}^{(i)} \in C_j} \mathbf{x}^{(i)}.$$

k 均值算法的目标是最小化簇内平方和（within-cluster sum of squares, WCSS）：

$$J = \sum_{j=1}^k \sum_{\mathbf{x}^{(i)} \in C_j} \|\mathbf{x}^{(i)} - \boldsymbol{\mu}_j\|^2.$$

k 均值聚类通常采用迭代优化策略，其过程可以概括为以下几个步骤：

- 1) 初始化：随机选择 k 个样本作为初始簇中心 $\boldsymbol{\mu}_1, \dots, \boldsymbol{\mu}_k$ 。
- 2) 分配步骤（Assignment step）：对每个样本 $\mathbf{x}^{(i)}$ ，将其分配给最近的簇中心，即

$$\mathbf{x}^{(i)} \in C_j \quad \text{若} \quad \|\mathbf{x}^{(i)} - \boldsymbol{\mu}_j\| \leq \|\mathbf{x}^{(i)} - \boldsymbol{\mu}_l\|, \forall l.$$

- 3) 更新步骤（Update step）：根据新的簇划分，重新计算每个簇的中心：

$$\boldsymbol{\mu}_j = \frac{1}{|C_j|} \sum_{\mathbf{x}^{(i)} \in C_j} \mathbf{x}^{(i)}.$$

- 4) 重复步骤 2) 和 3)，直到簇划分不再发生变化，或目标函数 J 收敛到某个阈值为止。

k 均值聚类具有算法简单、计算效率高的特点，适合大规模数据分析。当簇呈近似球形且规模相近时，其效果往往较好。然而，该方法需要预先指定簇数 k ，对初始簇中心敏感，容易陷入局部最优。同时，k 均值假设簇为凸形，不适合处理复杂分布或大小差异明显的簇，并且对噪声与离群点较为敏感。

利用自编码器进行聚的一种简单的思路是，先将样本 $\mathbf{x}^{(i)}$ 经过编码器得到潜在表示：

$$\mathbf{z}^{(i)} = f_{\theta}(\mathbf{x}^{(i)}).$$

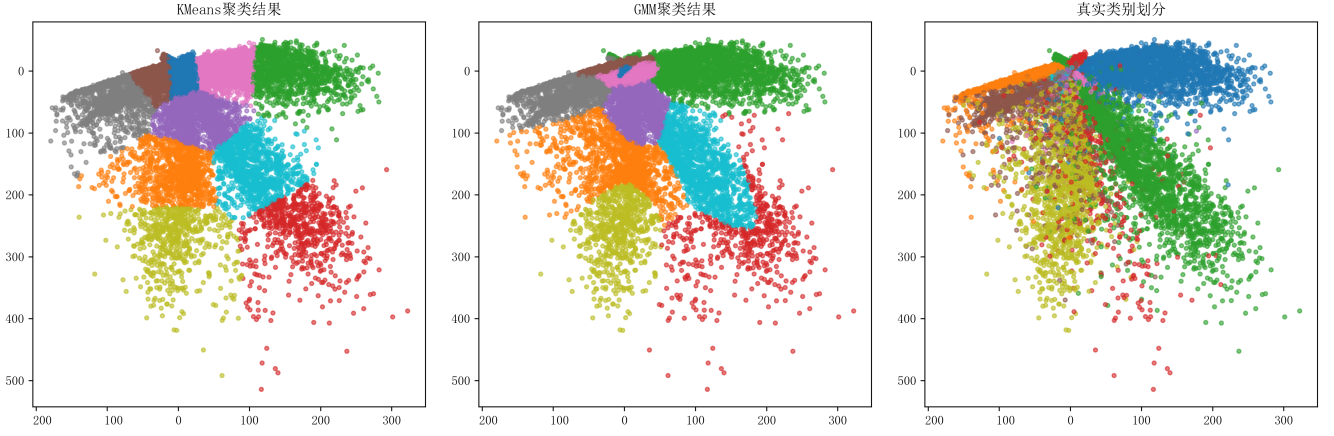
然后在 $\{\mathbf{z}^{(i)}\}$ 空间上执行聚类算法（k 均值或其他聚类算法）：

$$\min_{\{C_j\}} \sum_{i=1}^N \|\mathbf{z}^{(i)} - \boldsymbol{\mu}_{c(i)}\|^2,$$

其中 $\boldsymbol{\mu}_{c(i)}$ 是所属簇中心。

这种“降维 + 聚类”的组合往往比在原始高维空间直接聚类效果更好。

- **实例：** 05-ae_kmeans_gmm.py



(2) 深度聚类

在高维、非线性的数据空间中，传统的聚类方法（如 **k-means**）往往表现有限，因为欧氏距离在高维空间里难以有效衡量样本间的实际相似性。为此，**深度嵌入聚类**（Deep Embedded Clustering, DEC）提出了一种结合表示学习与聚类优化的框架，通过学习一个适合聚类的潜在表示空间来提升聚类效果。

DEC 首先利用 **自编码器** 或其他非线性映射将数据嵌入到低维潜在空间，然后在该空间中引入聚类目标进行优化。与传统先降维再聚类的方法不同，DEC 在预训练基础上继续通过聚类目标函数微调编码器参数，使得潜在表示更符合聚类结构。

DEC 模型主要包含两个部分：

- **编码器网络** 编码器将原始样本 $\mathbf{x}^{(i)}$ 映射为低维潜在表示 $\mathbf{z}^{(i)}$ ：

$$\mathbf{z}^{(i)} = f_{\theta}(\mathbf{x}^{(i)})$$

其中 f_{θ} 是由深度神经网络参数化的非线性映射。

- **聚类层** 在潜在空间中引入 K 个簇中心 $\boldsymbol{\mu}_j (j = 1, \dots, K)$ 相似度，得到软分配概率：

$$q_{ij} = \frac{(1 + \|\mathbf{z}^{(i)} - \boldsymbol{\mu}_j\|^2 / \alpha)^{-\frac{\alpha+1}{2}}}{\sum_{j'} (1 + \|\mathbf{z}^{(i)} - \boldsymbol{\mu}_{j'}\|^2 / \alpha)^{-\frac{\alpha+1}{2}}}$$

其中 α 是自由度参数（通常取 1）， q_{ij} 表示样本 i 属于簇 j 的概率。

DEC 通过最小化 **KL 散度** 来逐步提高聚类质量。为了增强聚类的置信度，DEC 定义了一个目标分布 p_{ij} ：

$$p_{ij} = \frac{q_{ij}^2 / \sum_i q_{ij}}{\sum_{j'} (q_{ij'}^2 / \sum_i q_{ij'})}$$

该目标分布对高置信度的分配赋予更大权重，从而加速簇的收敛。DEC 通过最小化 KL 散度来优化聚类层与编码器参数：

$$\mathcal{L} = \text{KL}(P \| Q) = \sum_i \sum_j p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

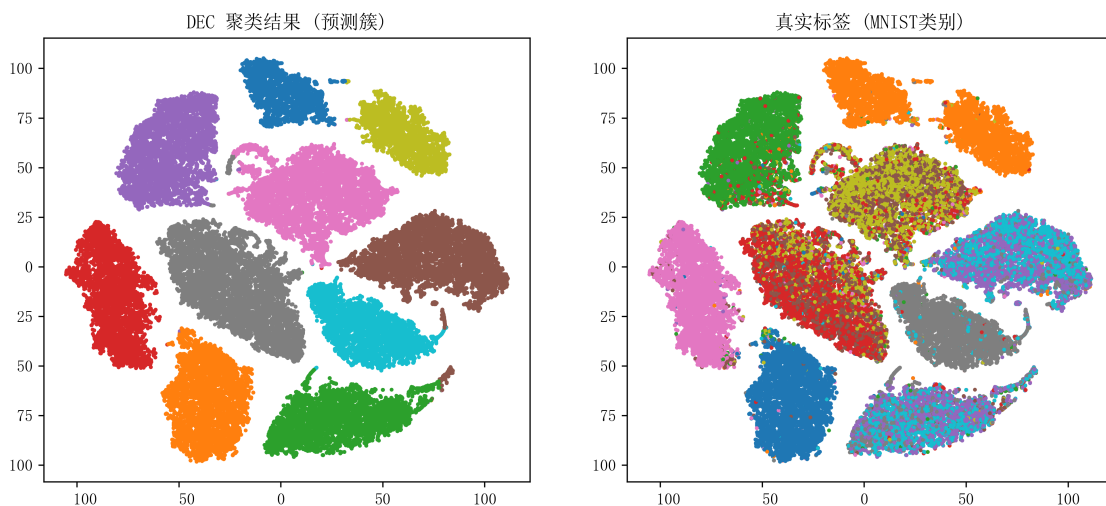
DEC 的训练分为两个阶段：

- **自编码器预训练** 使用重构误差训练自编码器，得到较好的初始潜在表示。此时编码器能够捕捉数据的主要结构信息。
- **聚类微调**
 - 丢弃解码器，仅保留编码器。
 - 在潜在表示空间运行 **k-means** 来初始化簇中心。
 - 继续更新编码器参数和簇中心，通过最小化 KL 散度优化，使潜在空间逐渐适应聚类需求。

在 DEC 中，每个样本在潜在空间中会被分配到每个簇的概率，也就是软分配 q_{ij} 。初始的潜在空间可能比较混乱，样本并没有明显的簇结构。为了让聚类效果更好，DEC 引入了一个“理想分布” p_{ij} ，它会放大每个样本最可能属于的簇的权重，同时平衡不同簇的影响。在训练时，DEC 的目标是让实际的软分配 q_{ij} 尽量接近目标分布 p_{ij} ，也就是最小化 KL 散度。这个过程可以理解为：每个样本都会被“拉向”它最可能属于的簇中心，而簇中心也会根据样本不断调整位置。随着训练进行，样本慢慢聚集在各自的簇周围，簇与簇之间也越来越分开。

DEC 并非严格的端到端模型，而是“**预训练 + 微调**”的两阶段方法。其关键贡献在于将深度表示学习与聚类目标结合，使学习到的潜在空间更加适合聚类任务，从而在高维、非线性数据上显著优于传统方法。

- **实例**：06-dec.py



4.3 异常检测

异常检测（Anomaly Detection）是自编码器的经典应用。其原理是自编码器在训练过程中只能学会重建“正常模式”的数据，对于异常数据则难以重建，导致重建误差显著增大。

给定输入 $\mathbf{x}$ ，定义重建误差：

$$e(\mathbf{x}) = \|\mathbf{x} - \hat{\mathbf{x}}\|_2^2.$$

如果 $e(\mathbf{x})$ 超过某个阈值 τ ，则判定该样本为异常：

$$\text{Anomaly}(\mathbf{x}) = \begin{cases} 1, & e(\mathbf{x}) > \tau \\ 0, & e(\mathbf{x}) \leq \tau \end{cases}.$$

DAGMM (Deep Autoencoding Gaussian Mixture Model) 是这种思路的改进，通过引入高斯混合模型使得模型能够估计异常样本的概率。并且，DAGMM还是一个端到端的模型。

(1) DAGMM模型组成

DAGMM 由 **压缩网络 (Compression Network)** 和 **估计网络 (Estimation Network)** 组成。

压缩网络类似传统自编码器，由编码器 f_{enc} 和解码器 f_{dec} 构成。给定输入样本 $\mathbf{x}$ ，编码器将其映射为 d 维潜在表示 $\mathbf{z}_c$ ：

$$\mathbf{z}_c = f_{\text{enc}}(\mathbf{x})$$

解码器将潜在表示重构为 $\hat{\mathbf{x}}$ ：

$$\hat{\mathbf{x}} = f_{\text{dec}}(\mathbf{z}_c)$$

为了增强异常检测能力，DAGMM 还计算重构误差和余弦相似度两个额外特征。具体地，计算重构误差（欧氏距离）和输入与重构向量之间的余弦相似度。将这些特征与潜在表示 $\mathbf{z}_c$ 拼接，形成最终输入估计网络的向量 $\mathbf{z} \in \mathbb{R}^{d+2}$ ：

$$\mathbf{z} = [\mathbf{z}_c, \|\mathbf{x} - \hat{\mathbf{x}}\|_2, \cos(\mathbf{x}, \hat{\mathbf{x}})]$$

这里，重构误差和余弦相似度为潜在表示提供额外的异常指示信息，使模型不仅能捕捉样本主要特征，还能量化其偏离正常模式的程度。

估计网络将 $\mathbf{z}$ 输入一个小型全连接网络，输出 K 个高斯分量的软分配 γ_k ：

$$\gamma_k = \text{softmax}(f_{\text{est}}(\mathbf{z}))_k$$

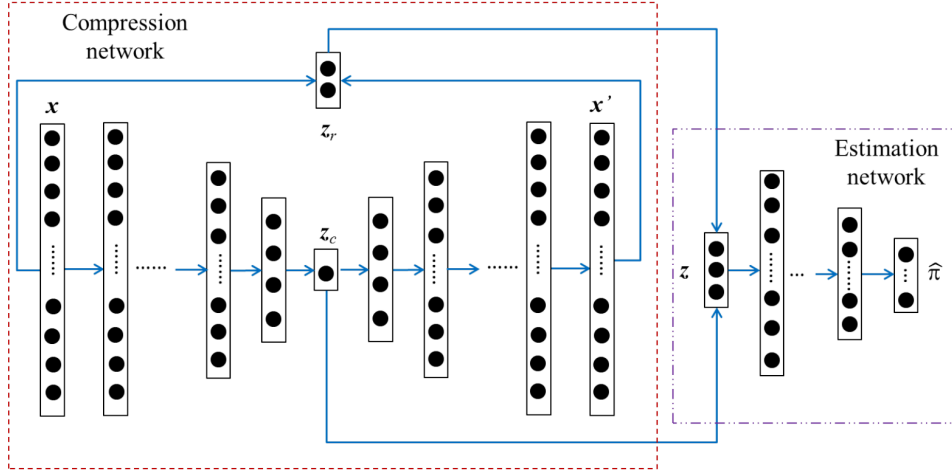
同时，模型利用 mini-batch 中所有样本的软分配估计混合高斯模型 (GMM) 的参数，包括混合系数 π_k 、均值向量 μ_k 和协方差矩阵 Σ_k ：

$$\begin{aligned}\pi_k &= \frac{1}{N} \sum_{i=1}^N \gamma_k^{(i)} \\ \mu_k &= \frac{\sum_{i=1}^N \gamma_{ik} \mathbf{z}^{(i)}}{\sum_{i=1}^N \gamma_k^{(i)}} \\ \Sigma_k &= \frac{\sum_{i=1}^N \gamma_{ik} (\mathbf{z}^{(i)} - \mu_k)(\mathbf{z}^{(i)} - \mu_k)^\top}{\sum_{i=1}^N \gamma_k^{(i)}}\end{aligned}$$

从而定义潜在向量 $\mathbf{z}$ 的概率密度：

$$p(\mathbf{z}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{z} | \mu_k, \Sigma_k)$$

GMM 模型通过多个成分的线性组合可以精确描述多簇结构，使正常样本在潜在空间形成多个高密度簇，而异常样本由于偏离这些簇，自然落在低密度区域。GMM 同时提供了概率密度估计，使异常检测不仅依赖重构误差，还能基于潜在表示的低概率密度判断异常。这种概率密度约束也引导压缩网络学习更结构化的潜在表示，使正常样本聚集而异常样本远离高密度簇。



(2) 训练目标

DAGMM 的训练目标由 **重构误差损失** 和 **GMM 最大似然损失** 构成，并加入正则化项防止协方差退化。

压缩网络的重构误差采用均方误差（MSE）：

$$\mathcal{L}_{\text{recon}} = \frac{1}{N} \sum_{i=1}^N \|\mathbf{x}^{(i)} - \hat{\mathbf{x}}^{(i)}\|^2$$

对低维向量 $\mathbf{z}^{(i)}$ 建立 GMM 后，其对数似然为：

$$\log p(\mathbf{z}^{(i)}) = \log \left(\sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{z}^{(i)} | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \right)$$

GMM 损失定义为负对数似然的平均：

$$\mathcal{L}_{\text{GMM}} = -\frac{1}{N} \sum_{i=1}^N \log \left(\sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{z}^{(i)} | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \right)$$

为了避免协方差矩阵退化（变得奇异或接近奇异），加入正则化项通过约束协方差矩阵的迹来保持其数值稳定性：

$$\mathcal{L}_{\text{reg}} = \lambda \sum_{k=1}^K \text{tr}(\boldsymbol{\Sigma}_k)$$

综合以上三部分，DAGMM 的总损失函数为：

$$\mathcal{L} = \mathcal{L}_{\text{recon}} + \mathcal{L}_{\text{GMM}} + \mathcal{L}_{\text{reg}}$$

模型通过端到端训练，压缩网络和估计网络同时优化，使潜在表示既能重构输入，又能符合 GMM 的概率分布。

(3) 异常检测方法

在测试阶段，对每个样本计算 **能量分数 (Sample Energy)**，用于异常判别：

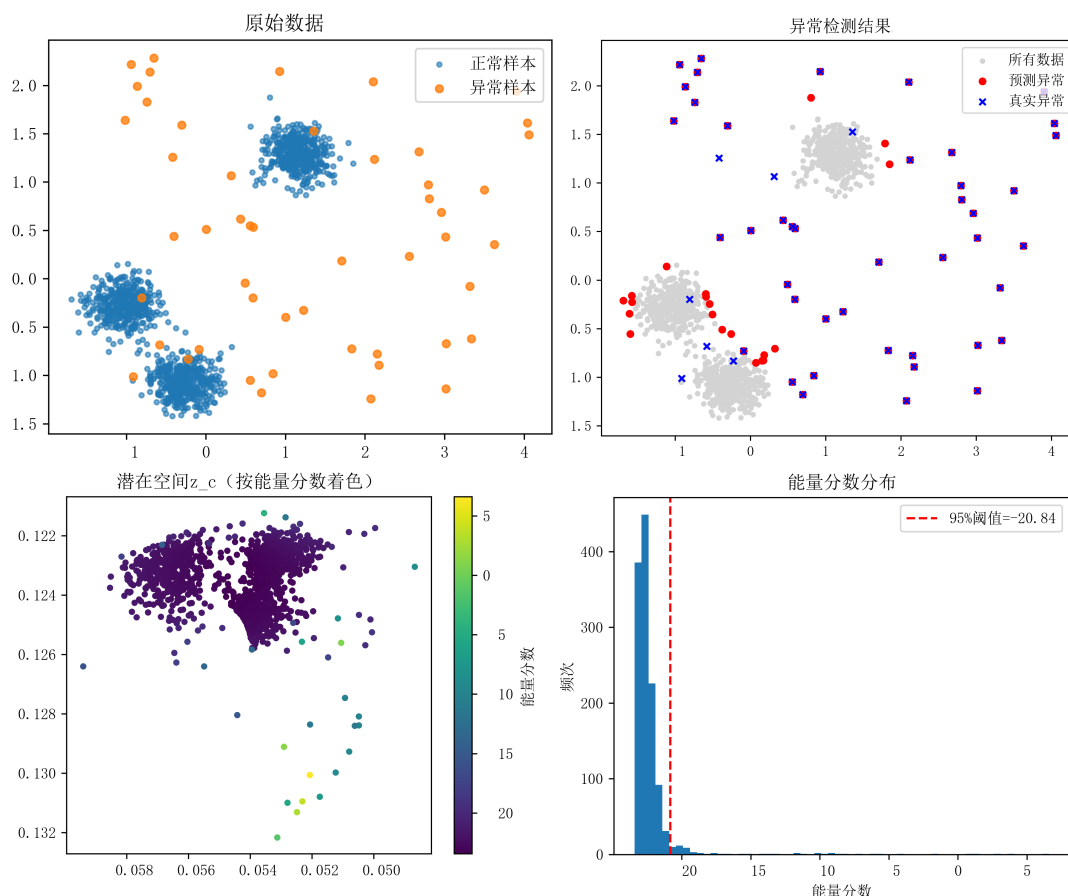
$$E(\mathbf{x}) = -\log \left(\sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{z} | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \right)$$

能量分数越高，说明样本在潜在分布下概率越低，越有可能是异常。通过设定阈值 $E(\mathbf{x}) > \tau$ ，即可实现异常判定。相比传统自编码器方法，DAGMM 不仅考虑重构误差，还结合潜在空间概率分布，使检测更加鲁棒且解释性强。

混合高斯分布能够精确建模正常数据的多模态结构，使不同类型的正常样本形成多个高密度簇，而异常样本由于偏离这些簇，被分配到低密度区域，获得高能量分数。同时，这种分布约束引导潜在空间学习更加结构化，增强异常检测能力。

(4) 实例

07-dagmm.py



4.4 推荐系统

推荐系统 (Recommender System, RS) 是一类用于从大量信息中预测用户兴趣并提供个性化建议的系统。它广泛应用于电子商务平台、流媒体平台、社交媒体以及新闻推荐等领域。推荐系统的核心任务是根据用户的历史行为 (如评分、点击、购买等) 预测用户对未接触项目的兴趣，从而实现个性化推荐。推荐系统主要可分为如下类型：

- **基于内容的推荐方法**利用项目自身的特征或属性来推荐与用户历史偏好相似的项目，例如，根据用户历史喜欢的电影类型、导演或演员来推荐新电影。此类方法在处理冷启动项目时表现较好，因为即使没有用户评分信息，也可以根据内容特征进行推荐，但它难以捕捉用户潜在的复杂偏好模式，容易出现推荐的同质化问题。
- **协同过滤（Collaborative filtering）方法**依赖于用户与项目之间的历史交互信息来进行推荐，其基本假设是相似用户喜欢相似项目，或相似项目被相似用户喜欢。协同过滤可以直接利用用户行为数据，而无需项目内容特征，因此在捕捉用户隐性偏好方面表现更好。
 - **基于用户的 CF**：找到兴趣相似的用户，推荐他们喜欢的项目。
 - **基于项目的 CF**：找到相似的项目，推荐给用户。
- **混合推荐方法**将基于内容和协同过滤的方法相结合，通过整合用户行为信息和项目内容特征，既能够缓解冷启动问题，又能够捕捉用户复杂的非线性偏好模式。

其中，协同过滤将推荐问题构建为一个矩阵补全问题。该矩阵分别以用户和物品为行和列，矩阵元素为用户对物品的评分。协同过滤通过预测该矩阵缺失的部分来实现推荐。不过，这种方法面临评分矩阵稀疏和冷启动问题。尤其在低维稀疏环境中，传统协同过滤难以学习用户复杂的非线性偏好模式。

下面介绍一种基于自编码器的混合推荐模型。

(1) 问题描述

给定一个用户-项目评分矩阵 $\mathbf{R} \in \mathbb{R}^{m \times n}$ ，其中 m 是用户数， n 是项目数，矩阵中的元素 r_{ui} 表示用户 u 对项目 i 的评分（缺失值表示未评分）。目标是预测缺失评分，从而进行个性化推荐。

传统的矩阵分解方法通过低秩分解：

$$\mathbf{R} \approx \mathbf{U}\mathbf{V}^T$$

其中， $\mathbf{U} \in \mathbb{R}^{m \times k}$ 和 $\mathbf{V} \in \mathbb{R}^{n \times k}$ 分别是用户和项目的潜在因子矩阵， k 是潜在因子的维度。

(2) 基于自编码器的推荐模型

模型以每个用户 u 的评分向量 $\mathbf{r}_u \in \mathbb{R}^n$ 作为自编码器的输入，并且对其进行掩码（mask）以填充未知数据，并避免对模型的学习产生误导。

模型的**编码器（Encoder）**表示为：

$$\mathbf{z}_u = f_\theta(\mathbf{r}_u) = \sigma(\mathbf{W}_e \mathbf{r}_u + \mathbf{b}_e)$$

其中， $\mathbf{z}_u \in \mathbb{R}^k$ 是潜在表示向量， σ 是非线性激活函数（如 ReLU 或 Sigmoid）， $\mathbf{W}_e \in \mathbb{R}^{k \times n}$ 和 $\mathbf{b}_e \in \mathbb{R}^k$ 分别是编码器的权重和偏置。

模型的**解码器（Decoder）**表示为：

$$\hat{\mathbf{r}}_u = g_\phi(\mathbf{z}_u) = \sigma(\mathbf{W}_d \mathbf{z}_u + \mathbf{b}_d)$$

其中， $\hat{\mathbf{r}}_u \in \mathbb{R}^n$ 是重构的评分向量， $\mathbf{W}_d \in \mathbb{R}^{n \times k}$ 和 $\mathbf{b}_d \in \mathbb{R}^n$ 分别是解码器的权重和偏置。

在**损失函数**中，为了适应输入数据中的缺失值，仅对已评分项使用均方误差（MSE）计算重构误差：

$$\mathcal{L}_u = \sum_{i \in \Omega_u} (\hat{r}_{ui} - r_{ui})^2$$

其中， Ω_u 表示用户 u 已评分的项目集合。

模型型还可以通过融合内容信息和协同信息，来提升性能并改善冷启动问题。该自编码器可通过去噪自编码器实现，并将项目信息（如类别、标签）或用户特征（如年龄、性别等）与评分向量一同输入自编码器。

训练目标是最小化重构误差，同时加入正则化防止过拟合：

$$\mathcal{L} = \sum_{u=1}^m \sum_{i \in \Omega_u} (\hat{r}_{ui} - r_{ui})^2 + \lambda (\|\mathbf{W}_e\|_F^2 + \|\mathbf{W}_d\|_F^2)$$

其中， λ 是正则化系数， $\|\cdot\|_F$ 表示 Frobenius 范数。训练可以使用梯度下降或 Adam 优化器。

(3) 实例

08-ae_cf.py

5. 小结

自编码器作为一种典型的 **无监督学习模型**，在机器学习和深度学习的发展中占据着重要地位。其基本思想是通过编码器-解码器结构，将高维数据映射到低维潜在空间，再从潜在空间中重构原始数据。这一过程不仅实现了对数据的压缩表示，还为我们理解和操作复杂数据提供了新的途径。

在最基本的形式中，自编码器仅仅通过最小化重构误差来学习数据的主要特征。然而，随着研究的深入，学者们提出了多种变体，如不完备自编码器、稀疏自编码器和去噪自编码器。这些变体分别通过降低潜在空间维度、增加稀疏性约束或引入噪声鲁棒性，拓展了自编码器的建模能力，使其能够更好地应对不同任务和数据分布。

在应用层面，自编码器展现出了广泛的适用性。作为降维工具，它为高维数据分析提供了一种非线性替代方法，能够比 PCA 更好地捕捉复杂流形结构；在聚类任务中，自编码器通过学习表征空间提升了聚类的效果，推动了深度聚类的发展；在异常检测中，自编码器利用重构误差对偏离正常模式的样本进行识别，在金融风控和工业监测等领域发挥了重要作用；在推荐系统中，自编码器通过深度表示学习刻画用户与物品之间的非线性关系，成为协同过滤的新方法；在图像去噪与生成任务中，自编码器展现出优异的鲁棒性和生成能力，为后续的生成模型奠定了实验与理论基础。

未来，自编码器及其扩展形式仍将在多个方向继续发挥作用。一方面，随着数据规模与复杂度的不断提升，自编码器将在非监督表征学习与下游任务结合中展现更大价值；另一方面，结合自监督学习、对比学习以及大规模生成模型的研究趋势，自编码器有望在可解释性、稳健性和跨模态建模方面获得进一步突破。

参考文献

- 自编码器：Reducing the Dimensionality of Data with Neural Networks, 2006.
- 稀疏自编码器：<https://web.stanford.edu/class/cs294a/sparseAutoencoder.pdf>
- 去噪声自编码器：Extracting and composing robust features with denoising autoencoders. 2008.

- 深度聚类: **Unsupervised Deep Embedding for Clustering Analysis**, 2015.
- 异常检测: **Deep autoencoding gaussian mixture model for unsupervised anomaly detection**, 2018.
- 信息推荐: **Hybrid Recommender System based on Autoencoders**, 2016.